Rapport de test d'intrusion

Projet final — Cyber Challenge

Red Team

Auteur	Romain LIGNERES
Filière	Cyber
Binôme	Arthur LAUZANNE - Blue team
École	ECE Paris
Année	2025-2026
Date	5 avril 2026

Table des matières

1 Introduction **3**

2 Périmètre **3**

3 Méthodologie **3**

4 Vulnérabilité 1 — Injection SQL **4**
4.1 Localisation 4
4.2 Description 4
4.3 Exploitation 4
4.4 Preuve 4
4.5 Impact 5
4.6 Recommandation 5

5 Vulnérabilité 2 — XSS réfléchi **6**
5.1 Localisation 6
5.2 Description 6
5.3 Exploitation 6
5.4 Preuve 6
5.5 Impact 7
5.6 Recommandation 7

6 Vulnérabilité 3 — XSS stocké **8**
6.1 Localisation 8
6.2 Description 8
6.3 Exploitation 8
6.4 Preuve 8
6.5 Impact 8
6.6 Recommandation 8

7 Vulnérabilité 4 — IDOR (Insecure Direct Object Reference) **9**
7.1 Localisation 9
7.2 Description 9
7.3 Exploitation 9
7.4 Preuve 9
7.5 Impact 9
7.6 Recommandation 9

8 Vulnérabilité 5 — Path Traversal **10**
8.1 Localisation 10
8.2 Description 10
8.3 Exploitation 10
8.4 Preuve 10
8.5 Impact 10

8.6 Recommandation 10

9 Vulnérabilité 6 — Broken Access Control 11
9.1 Localisation 11
9.2 Description 11
9.3 Exploitation 11
9.4 Preuve 11
9.5 Impact 11
9.6 Recommandation 12

10 Vulnérabilité 7 — Divulgation d'informations 13
10.1 Localisation 13
10.2 Description 13
10.3 Exploitation 13
10.4 Preuve 13
10.5 Impact 13
10.6 Recommandation 13

11 Vulnérabilité 8 — Open Redirect 14
11.1 Localisation 14
11.2 Description 14
11.3 Exploitation 14
11.4 Preuve 14
11.5 Impact 14
11.6 Recommandation 14

12 Vulnérabilité 9 — OS Command Injection 15
12.1 Localisation 15
12.2 Description 15
12.3 Exploitation 15
12.4 Preuve 15
12.5 Impact 15
12.6 Recommandation 15

13 Conclusion 16

Introduction

Ce rapport documente les tests d'intrusion menés sur la façade Moodle développée par la Blue Team dans le cadre du projet final du cours Sécurité des SI. L'objectif était d'identifier et d'exploiter des vulnérabilités web afin de compromettre l'application, puis de formuler des recommandations concrètes pour y remédier. L'application cible tourne en local.

Périmètre

Élément	**Détail**
Application	Façade Moodle simplifiée (Blue Team)
URL	`http://localhost:3000`
Technologies	Node.js, Express.js, SQLite
Pages testées	`/login`, `/calendar`, ...

Méthodologie

Les tests ont suivi trois étapes classiques en pentest applicatif :

1. **Reconnaissance** — analyse du comportement de l'application via le navigateur, inspection du code source côté client, identification des points d'entrée.
2. **Exploitation** — envoi de payloads manuels sur les champs identifiés, observation des réponses du serveur.
3. **Documentation** — capture des preuves, rédaction du présent rapport.

Vulnérabilité 1 — Injection SQL

Localisation

Route `POST /login` — traitement du formulaire de connexion.

Description

Lors de mes tests sur le formulaire de connexion, j'ai d'abord tenté d'insérer une apostrophe simple dans le champ username. Le serveur a retourné une erreur de base de données, ce qui m'a confirmé que mon entrée était directement interprétée dans la requête SQL sans aucun filtrage. J'ai ensuite utilisé un payload classique d'injection SQL pour contourner l'authentification.

Exploitation

Dans le champ *Nom d'utilisateur*, j'ai saisi le payload suivant :

```
' OR '1'='1'--
```

Avec n'importe quelle valeur dans le champ mot de passe, la requête exécutée devient :

```
SELECT *
FROM users
WHERE username = '' OR '1'='1'--'
AND password = ''
```

La condition `'1'='1'` étant toujours vraie, le serveur retourne le premier utilisateur de la base et accorde l'accès sans vérification du mot de passe.

Preuve

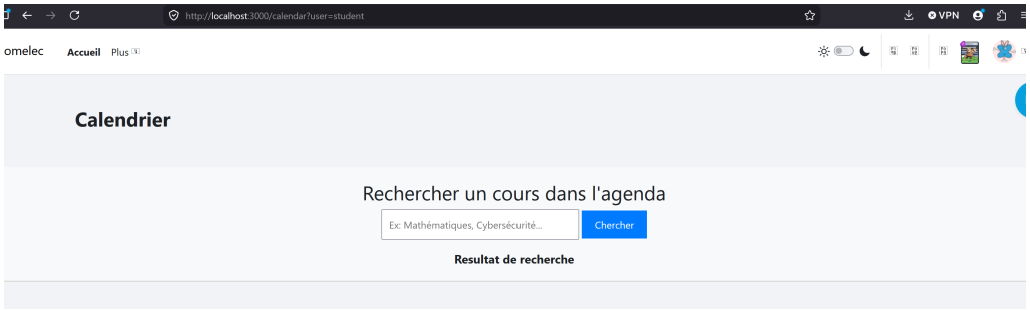

FIGURE 1 – Preuve de connexion sans connaître le mdp

Impact

Contournement total de l'authentification. Dans un contexte réel, un attaquant pourrait accéder à l'ensemble des fonctionnalités réservées aux utilisateurs connectés.

Recommandation

Utiliser des requêtes préparées avec paramètres liés :

```
db.get(
  "SELECT * FROM users WHERE username = ? AND password =
     ?",
  [username, password],
  callback
);
```

Vulnérabilité 2 — XSS réfléchi

Localisation

Route `GET /calendar` — paramètre d'URL `recherche`.

Description

En testant la barre de recherche, j'ai remarqué que ce que je tape apparaît directement dans la page et dans l'URL. J'ai alors testé si le site filtrait les balises HTML en injectant une balise <script>. Le navigateur a exécuté le code, confirmant la faille.

Exploitation

En naviguant vers l'URL suivante une fois connecté :

```
http://localhost:3000/calendar?recherche=
  <script>alert('XSS')</script>
```

Ce qui s'est passé. Une popup JavaScript s'est affichée avec le texte "XSS", prouvant que le code a été exécuté par le navigateur.

Preuve

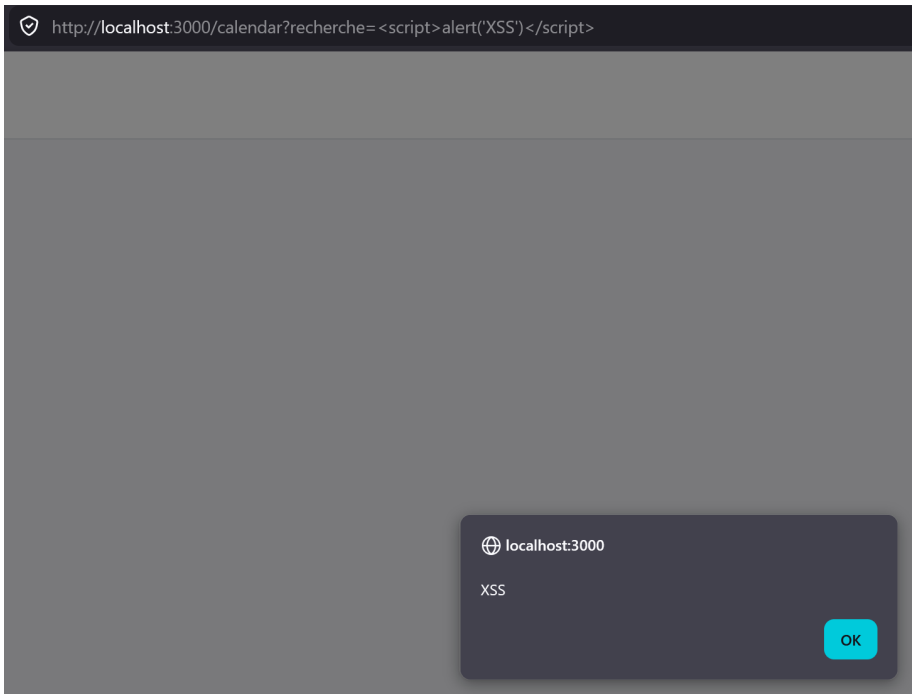

FIGURE 2 – Preuve d'injection de code malveillant

Impact

Dans un contexte réel, un attaquant pourrait remplacer le alert par du code qui vole le cookie de session d'un utilisateur et se connecter à sa place sans connaître son mot de passe.

Recommandation

Échapper les caractères spéciaux avant toute insertion dans le HTML :

```
const safe = motRecherche
  .replace(/&/g,  '&amp;')
  .replace(/</g,  '&lt;')
  .replace(/>/g,  '&gt;')
  .replace(/"/g,  '&quot;');
```

Vulnérabilité 3 — XSS stocké

Localisation

Route `GET /calendar?user=student`

Description

Lorsque j'ai voulu ajouter un commentaire en bas de page, je me suis rendu compte que je pouvais injecter du code avec des balises. Les balises HTML ne sont pas filtrées, ce qui permet d'injecter du code malveillant.

Exploitation

En naviguant vers l'URL suivante une fois connecté :

```
<b>teste</b>
```

Ce qui s'est passé. J'ai pu afficher le mot "test" en gras grâce au balise html ce qui prouve la défaillance du système.

Preuve

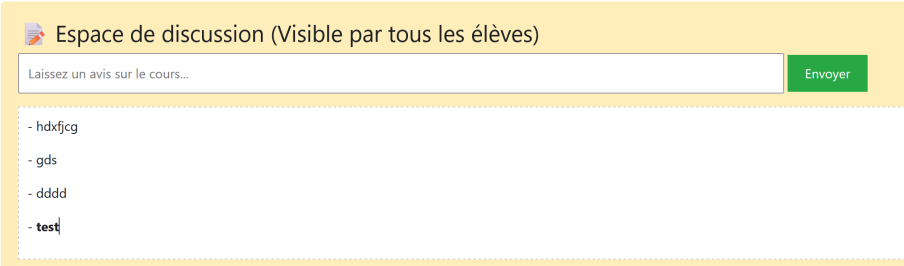

FIGURE 3 – Preuve d'injection de code malveillant dans l'espace de commentaire

Impact

Tout code HTML inséré dans les commentaires est sauvegardé en base de données et réaffiché à tous les visiteurs sans nettoyage. Contrairement au XSS réfléchi qui ne touche qu'une personne via un lien piégé, ici le payload est permanent et s'exécute automatiquement pour chaque utilisateur qui charge la page. Dans un contexte réel avec des protections navigateur moins strictes, un attaquant pourrait voler les sessions de tous les utilisateurs connectés en une seule injection

Recommandation

Nettoyer toutes les données utilisateur avant de les stocker en base de données et avant de les afficher. En Node.js on peut utiliser la librairie DOMPurify ou simplement échapper les caractères HTML dangereux comme <, >, " avant l'insertion

Vulnérabilité 4 — IDOR (Insecure Direct Object Reference)

Localisation

Route `GET /calendar?user=student`

Description

J'ai remarqué que l'url de connexion pour un élève et de la forme /calendar ?user=student, j'ai donc tenté de remplacer student par admin et je me suis retrouvé connecter en tant qu'admin.

Exploitation

Je me suis connecté via cette URL : /calendar ?user=admin

Preuve

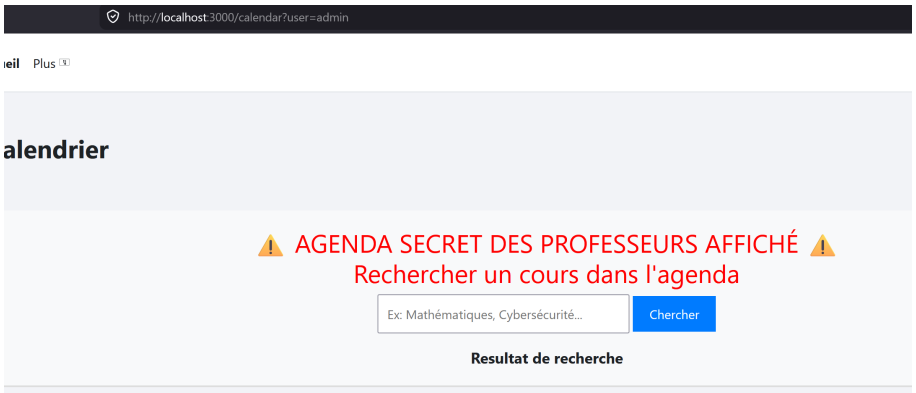

FIGURE 4 – Preuve de connexion à la page admin

Impact

Je me suis retrouvé connecté à la page admin sans connaître le mot de passe. Un hacker peut donc récuperer toutes les infos ou les modifier sur cette page.

Recommandation

La correction de cette faille se fait côte serveur en vérifiant les rôles via un systèmes de session.

Vulnérabilité 5 — Path Traversal

Localisation

Route `GET /download?`

Description

J'ai remarqué que l'url pouvait être modifié sans vérification. Le changement de l'URL permet de télécharger des nimporte quels fichiers du repo.

Exploitation

J'ai téléchagé le code du server.js via cet URL : /download ?file=server.js

Preuve

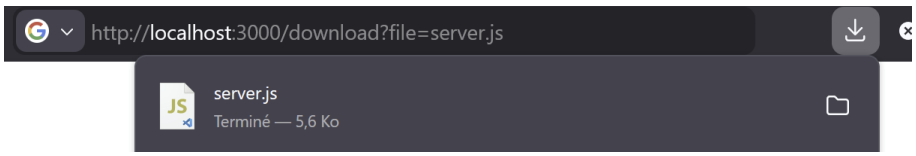

FIGURE 5 – Preuve de téléchargement du fichier server.js via la route /download

Impact

Le télechargement s'est éffectué donc j'ai accès à tout le code ou fichier que je veux. Plus aucune informations n'est confidentielle avce cette faille.

Recommandation

Le serveur doit vérifier que le fichier demandé se trouve bien dans le dossier autorisé et refuser tout chemin contenant .. ou des références à des fichiers sensibles.

Vulnérabilité 6 — Broken Access Control

Localisation

Route `GET /admin`

Description

J'ai modifié l'URL et me suis directement connecté sur une page admin qui contient des infos sensibles.

Exploitation

La page /admin est accessible sans aucune vérification d'authentification. N'importe qui connaissant l'URL peut y accéder directement

Preuve

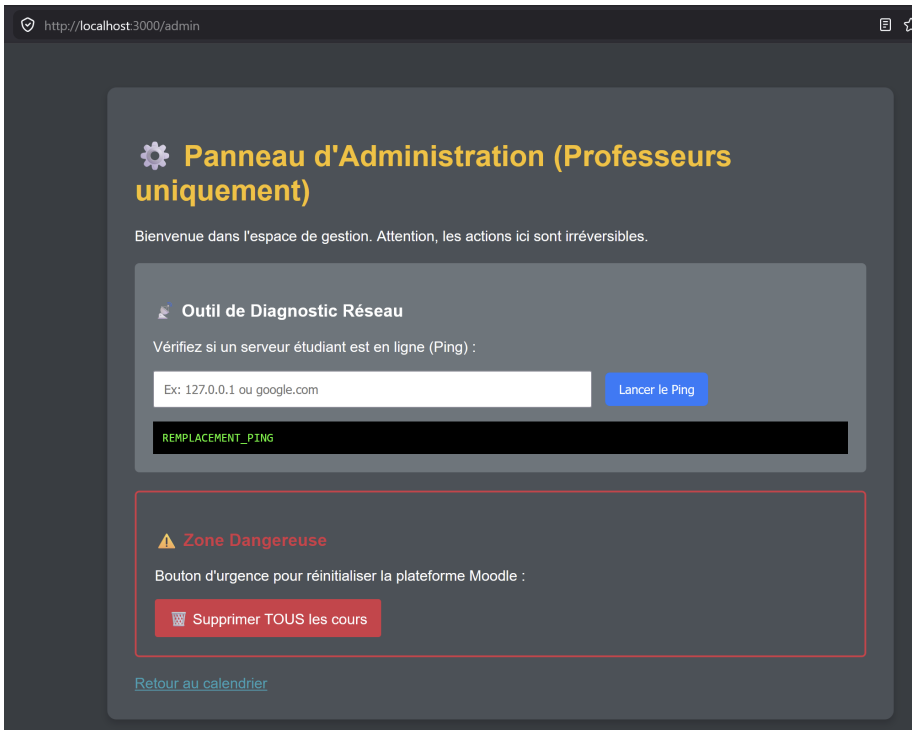

FIGURE 6 – Preuve de connexion à la page /admin

Impact

Un attaquant ayant accès à cette page peut supprimer toutes les données de la plateforme, utiliser l'outil de diagnostic réseau et réaliser des actions irréversibles normalement réservées aux administrateurs.

Recommandation

La correction consiste à vérifier côté serveur que l'utilisateur est connecté et possède le rôle admin avant d'afficher la page. Si ce n'est pas le cas, il doit être redirigé vers la page de connexion.

Vulnérabilité 7 — Divulgation d'informations

Localisation

Route `GET /debug/db`

Description

Une route de debug a été laissée accessible en production. En naviguant simplement vers /debug/db, n'importe qui peut accéder à l'intégralité de la base de données utilisateurs sans aucune authentification.

Exploitation

Accès direct via : http ://localhost :3000/debug/db

Preuve

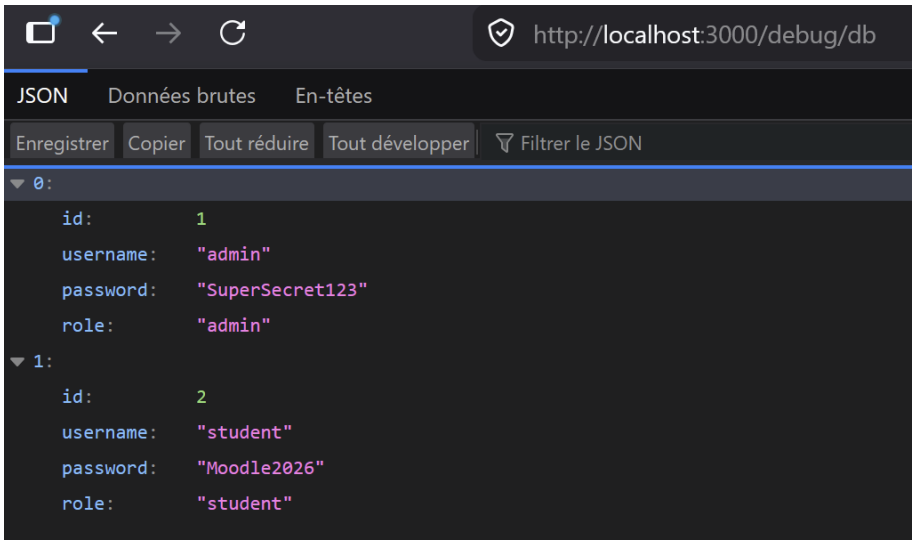

FIGURE 7 – Preuve de connexion à la BDD

Impact

Tous les identifiants et mots de passe des utilisateurs sont exposés en clair. Un attaquant peut se connecter avec n'importe quel compte sans avoir besoin de casser quoi que ce soit.

Recommandation

Supprimer toutes les routes de debug avant la mise en production. Les mots de passe doivent également être hashés et jamais stockés en clair.

Vulnérabilité 8 — Open Redirect

Localisation

Route POST /, paramètre next dans l'URL

Description

En analysant l'URL de la page de connexion, j'ai remarqué un paramètre next qui définit la redirection après connexion. Le site ne vérifie pas si cette URL appartient au même domaine..

Exploitation

En accédant à http ://localhost :3000/ ?next=https ://www.google.com et en me connectant normalement, j'ai été redirigé vers Google au lieu du calendrier.

Preuve

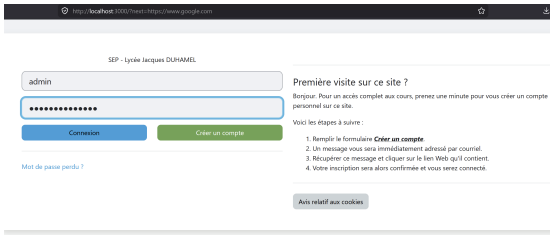

FIGURE 8 – Avant connexion

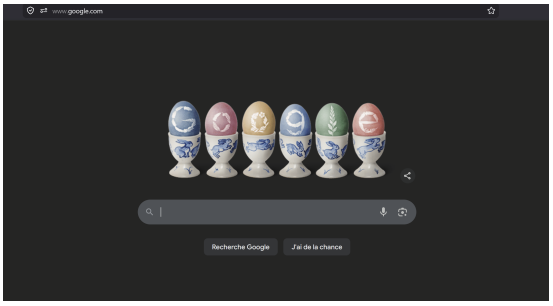

FIGURE 9 – Après connexion

Impact

Un attaquant peut envoyer ce lien piégé à une victime. Elle se connecte sur le vrai site puis est redirigée vers un faux site qui ressemble à Moodle. Elle pense devoir se reconnecter et entre ses identifiants qui sont alors volés.

Recommandation

Vérifier que l'URL de redirection appartient bien au même domaine avant de rediriger. Rejeter toute URL pointant vers un site externe.

Vulnérabilité 9 — OS Command Injection

Localisation

Route `POST /ping, champ IP de l'outil de diagnostic`

Description

L'outil de ping exécute directement l'entrée utilisateur dans le terminal du serveur sans aucune validation. En chaînant une commande avec & il est possible d'exécuter n'importe quelle commande système.

Exploitation

Nous avons utilisé le payload suivant : 127.0.0.1 & dir Le serveur a exécuté le ping puis la commande dir, affichant l'arborescence complète des fichiers du serveur dans la page.

Preuve

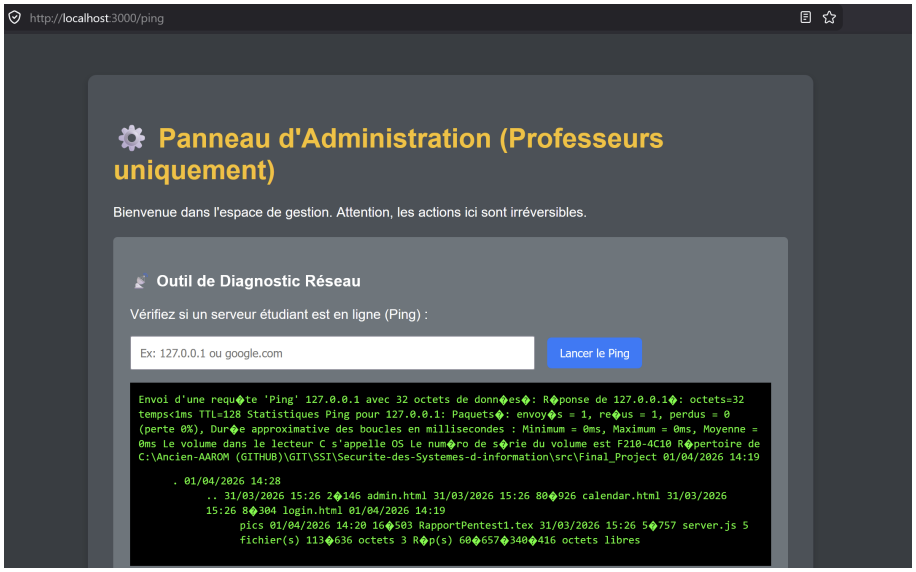

FIGURE 10 – Preuve d'OS injection

Impact

Un attaquant pourrait exécuter n'importe quelle commande sur le serveur, lire des fichiers sensibles, créer des utilisateurs, ou prendre le contrôle total de la machine.

Recommandation

Ne jamais exécuter directement les entrées utilisateur dans le terminal. Utiliser une liste blanche d'IPs valides et rejeter tout autre caractère.

Conclusion

Ce pentest a permis d'identifier et d'exploiter 9 vulnérabilités sur la façade Moodle développée par la Blue Team. Ces failles couvrent un large spectre des risques web les plus courants, plusieurs d'entre elles figurant dans l'OWASP Top 10.

#	Vulnérabilité	Criticité	Priorité
1	Injection SQL	Critique	Immédiate
2	XSS Réfléchi	Élevée	Haute
3	XSS Stocké	Élevée	Haute
4	IDOR	Élevée	Haute
5	Path Traversal	Critique	Immédiate
6	Broken Access Control	Élevée	Haute
7	Information Disclosure	Critique	Immédiate
8	Open Redirect	Moyenne	Normale
9	Command Injection	Critique	Immédiate

Les corrections prioritaires concernent les failles critiques : l'injection SQL, le Path Traversal, la divulgation d'informations et l'injection de commandes. Ces quatre failles donnent à un attaquant un accès quasi total au système.

Romain LIGNERES — Red Team — ECE Paris 2025-2026